

Теория графов и алгоритмы

Определения, представления, базовые и сложные алгоритмы

Неориентированные и ориентированные графы

Неориентированный граф

$G = (V, E)$, где V — множество вершин, $E \subseteq \{\{u, v\} \mid u, v \in V\}$ — множество неупорядоченных пар.

Ориентированный граф (орграф)

$D = (V, A)$, где $A \subseteq V \times V$ — множество упорядоченных пар (дуг).

Примеры:

- Неориентированный: дороги между городами, социальные сети (дружба).
- Ориентированный: ссылки на веб-страницах, граф вызовов функций.

Представления графов

Матрица смежности A размера $|V| \times |V|$:

$$A_{ij} = \begin{cases} 1 & \text{есть ребро } i \rightarrow j \\ 0 & \text{иначе} \end{cases}$$

Память: $O(V^2)$. Хорошо для плотных графов.

Матрица инцидентности B ($|V| \times |E|$):

- Неориент.: $B_{ve} = 1$, если v инцидентна e .
- Ориент.: $B_{ve} = 1$ (начало), -1 (конец), 0 иначе.

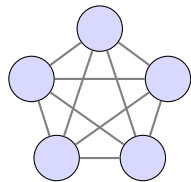
Матрица Кирхгофа (Лапласиан) L :

$$L_{ii} = \deg(v_i), \quad L_{ij} = -1 \text{ при } i \neq j \text{ и ребре } (i, j)$$

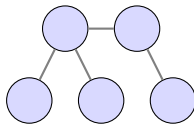
Список смежности: Массив списков соседей.
Память: $O(V + E)$. Лучше для разреженных.

- **Полный граф** K_n : все пары вершин соединены.
- **Дерево**: связный граф без циклов.
- **Двудольный граф**: $V = V_1 \cup V_2$, рёбра только между долями. $K_{m,n}$ — полный двудольный.
- **Остовное дерево**: подграф-дерево, содержащее все вершины.
- **Клика**: полный подграф.
- **Независимое множество**: вершины без рёбер между собой.

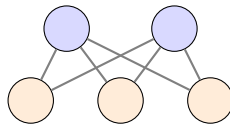
Иллюстрации: Основные виды графов



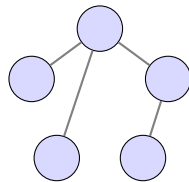
K_5 (полный)



Дерево



$K_{2,3}$ (двудольный)



Остовное дерево

Компоненты графов

Компонента связности (неориентированный граф)

Максимальный связный подграф. Две вершины находятся в одной компоненте, если существует путь между ними.

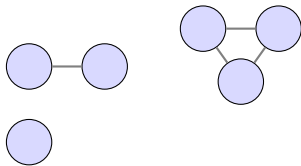
Компонента сильной связности (КСС) в ориентированном графе

Максимальный подграф, в котором для любой пары вершин u, v существует путь из u в v и из v в u .

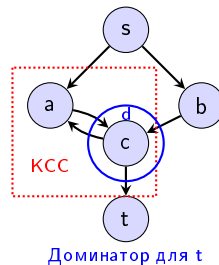
Вершина-доминатор (в орграфе с выделенным истоком s)

Вершина d называется **доминатором** вершины v ($d \neq v$), если любой путь из истока s в v проходит через d .

Компоненты связности (неор.)



КСС и доминатор (ор.)



Слева: Три компоненты связности. Справа: Орграф с истоком s ; вершины $\{a, b, c\}$ образуют КСС (розовая область). Вершина c – доминатор для t (все пути $s \rightarrow t$ проходят через c).

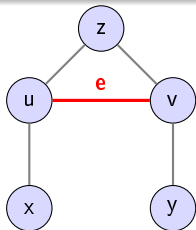
Операция стягивания ребра (edge contraction)

Определение

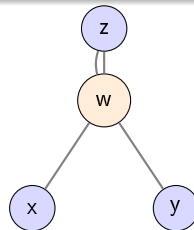
Стягивание ребра $e = \{u, v\}$ в графе G (обозначается G/e):

- 1 Удаляем ребро e .
- 2 Вершины u и v сливаем в одну новую вершину w .
- 3 Все рёбра, инцидентные u или v , теперь инцидентны w (петли удаляются, кратные рёбра можно оставить или заменить одним).

Полученный граф называется **минором** исходного графа.



стягиваем e



Ребро $e = \{u, v\}$ выделено красным

Новая вершина w (слияние u и v)

Утверждение

Для простого неориентированного графа G с матрицей смежности A и матрицей инцидентности B (строки — вершины, столбцы — рёбра, ориентация может быть выбрана произвольно) лапласиан можно выразить как

$$L = D - A = BB^T,$$

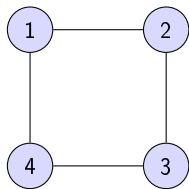
где D — диагональная матрица степеней вершин. Это подчёркивает, что лапласиан кодирует как структуру смежности, так и степени вершин.

Доказательство:

$$(BB^T)_{ii} = \sum_e B_{ie}^2 = \deg(v_i) = L_{ii}$$

$$(BB^T)_{ij} = \sum_e B_{ie}B_{je} = -1 \text{ (если ребро между } i \text{ и } j) = L_{ij}$$

Диаграмма: Лапласиан через инцидентность



$$A = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}$$

$$B = \begin{pmatrix} 1 & 0 & 0 & -1 \\ -1 & 1 & 0 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 0 & -1 & 1 \end{pmatrix}$$



$$L = BB^T = \begin{pmatrix} 2 & -1 & 0 & -1 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ -1 & 0 & -1 & 2 \end{pmatrix}$$

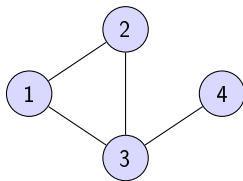
Рисунок показывает простой цикл C_4 . Ориентация рёбер выбрана по часовой стрелке. Лапласиан $L = BB^T$ восстанавливается независимо от выбора ориентации.

Матрица смежности и число путей

Утверждение

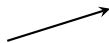
Пусть A — матрица смежности графа G . Тогда элемент $(A^k)_{ij}$ равен количеству путей длины ровно k из вершины i в вершину j . Это фундаментальное свойство позволяет анализировать связность и расстояния в графе с помощью матричных степеней.

Диаграмма: Степени матрицы смежности и пути



$$A = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

$$A^2 = \begin{pmatrix} 2 & 1 & 1 & 1 \\ 1 & 2 & 1 & 0 \\ 1 & 1 & 3 & 0 \\ 1 & 0 & 0 & 1 \end{pmatrix}$$



Пути длины 2:

$1 \rightarrow 3 \rightarrow 4$ даёт

$$(A^2)_{14} = 1$$

$1 \rightarrow 2 \rightarrow 1$ и

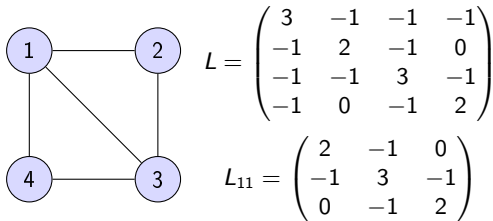
$1 \rightarrow 3 \rightarrow 1$ дают

$$(A^2)_{11} = 2$$

Теорема Кирхгофа (Kirchhoff's Theorem)

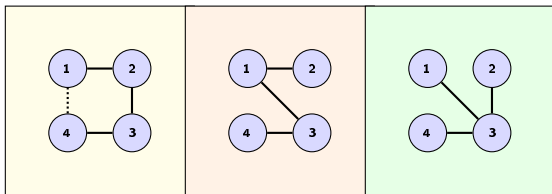
Для любого связного графа G с лапласианом L число остовных деревьев равно алгебраическому дополнению любого элемента L_{ij} (т.е. $|\det(L_{ij})|$ для любых i, j). Иными словами, любой кофактор лапласиана даёт количество остовных деревьев графа.

Диаграмма: Теорема Кирхгофа



$$\det(L_{11}) = 2(6 - 1) - (-1)(-2 - 0) = 10 - 2 = 8$$

8 остовных деревьев:



П-тип (4 шт.)

Z-тип (2 шт.)

Э-тип (2 шт.)

Граф “конверт” (4 вершины, 5 рёбер). Удаление вершины 1 и взятие определителя подматрицы L_{11} даёт 8 — количество остовных деревьев.

Algorithm 1 DFS (рекурсивный)

```
1: процедура DFS( $v$ )
2:    $visited[v] \leftarrow true$ 
3:   для each  $u \in neighbours(v)$  выполнять
4:     если not  $visited[u]$  то DFS( $u$ )
5:   конец если
6:   конец для
7: конец процедура
```

Применения:

- Поиск компонент связности.
- Проверка наличия циклов (по обратным рёбрам).
- Топологическая сортировка (время выхода).
- Поиск мостов и точек сочленения.

Сложность: $O(V + E)$.

Algorithm 2 BFS

```
1: процедура BFS( $s$ )
2:    $queue \leftarrow$  пустая очередь
3:    $enqueue(queue, s)$ 
4:    $dist[s] \leftarrow 0$ ,  $visited[s] \leftarrow true$ 
5:   пока  $queue$  не пуста выполнять
6:      $v \leftarrow dequeue(queue)$ 
7:     для  $each\ u \in neighbours(v)$  выполнять
8:       если  $not\ visited[u]$  то
9:          $visited[u] \leftarrow true$ 
10:         $dist[u] \leftarrow dist[v] + 1$ 
11:         $enqueue(queue, u)$ 
12:       конец если
13:     конец для
14:   конец пока
15: конец процедура
```

Union-Find (система непересекающихся множеств)

```
1: функция FIND( $x$ )
2:   если  $parent[x] \neq x$  то
3:      $parent[x] \leftarrow FIND(parent[x])$ 
4:   конец если
5:   вернуть  $parent[x]$ 
6: конец функция
7:
8: процедура UNION( $x, y$ )
9:    $rx \leftarrow FIND(x), ry \leftarrow FIND(y)$ 
10:  если  $rx = ry$  то
11:    вернуть
12:  конец если
13:  если  $rank[rx] < rank[ry]$  то
14:     $parent[rx] \leftarrow ry$ 
15:  иначе если  $rank[rx] > rank[ry]$  то
16:     $parent[ry] \leftarrow rx$ 
17:  иначе
18:     $parent[ry] \leftarrow rx$ 
19:     $rank[rx] \leftarrow rank[rx] + 1$ 
20:  конец если
21: конец процедура
```

▷ сжатие пути

Практически константное время ($\alpha(V)$ — обратная функция Аккермана).

Алгоритм Краскала (MST)

```
1: процедура KRUSKAL( $G = (V, E)$  с весами)
2:   Отсортировать  $E$  по возрастанию веса
3:   Инициализировать Union-Find для  $V$ 
4:    $mst \leftarrow \emptyset$ 
5:   для each  $(u, v, w) \in E$  в порядке сортировки выполнять
6:     если FIND( $u$ )  $\neq$  FIND( $v$ ) то
7:       UNION( $u, v$ )
8:        $mst \leftarrow mst \cup \{(u, v, w)\}$ 
9:     конец если
10:  конец для
11:  вернуть  $mst$ 
12: конец процедура
```

Жадный — добавляем наименьшее ребро, не образующее цикл. Сложность: $O(E \log E)$ (сортировка) + $O(E \cdot \alpha(V))$ (Union-Find).

Релаксация во взвешенных графах

Релаксация ребра — фундаментальная операция в алгоритмах на графах с весами.

Релаксация ребра (u, v) с весом $w(u, v)$: если текущее расстояние до v больше, чем расстояние до u плюс вес ребра, то обновляем $dist[v]$.

```
1: процедура RELAX( $u, v, w$ )  
2:   если  $dist[u] + w(u, v) < dist[v]$  то  
3:      $dist[v] \leftarrow dist[u] + w(u, v)$   
4:      $parent[v] \leftarrow u$   
5:   конец если  
6: конец процедуры
```

▷ восстановление пути

- Изначально $dist[s] = 0$, остальные $+\infty$.
- Каждая успешная релаксация уменьшает $dist[v]$.
- В невзвешенном графе — просто BFS.

Алгоритм Прима — подробно

Строим минимальное остовное дерево (MST), добавляя ближайшую к дереву вершину. Для MST вместо расстояния от старта используется минимальный вес ребра, соединяющего вершину с текущим деревом.

$$key[v] = \min_{(u,v) \in E, u \in tree} w(u, v)$$

```
1: процедура PRIM( $G, start$ )
2:   для each  $v \in V$  выполнять
3:      $key[v] \leftarrow \infty, parent[v] \leftarrow -1$ 
4:      $inQueue[v] \leftarrow true$ 
5:   конец для
6:    $key[start] \leftarrow 0$ 
7:    $pq \leftarrow$  мин-куча из всех вершин по ключу  $key$ 
8:   пока  $pq$  не пуста выполнять
9:      $v \leftarrow extractMin(pq)$ 
10:     $inQueue[v] \leftarrow false$ 
11:    для each  $u \in neighbours(v)$  выполнять
12:      если  $inQueue[u]$  and  $w(v, u) < key[u]$  to
13:         $key[u] \leftarrow w(v, u)$ 
14:         $parent[u] \leftarrow v$ 
15:         $decreaseKey(pq, u)$ 
16:      конец если
17:    конец для
18:  конец пока
19:  вернуть  $parent$ 
20: конец процедура
```

▷ v добавлена в MST

▷ релаксация для MST

Алгоритм Дейкстры — релаксация и корректность

Если веса неотрицательны, то первое извлечение вершины из кучи даёт окончательное кратчайшее расстояние.

- 1 Инициализация: $dist[s] = 0$, остальные ∞ .
- 2 Извлекаем вершину v с минимальным $dist[v]$ (ещё не обработанную).
- 3 Релаксируем все рёбра из v : для каждого соседа u пытаемся улучшить $dist[u]$.
- 4 Повторяем, пока все вершины не обработаны.

Algorithm 3 Дейкстра с явной релаксацией

```
1: процедура DIJKSTRA( $G, s$ )
2:    $dist[s] \leftarrow 0$ 
3:    $pq \leftarrow$  мин-куча ▷ ( $dist[v], v$ )
4:   пока  $pq$  не пуста выполнять
5:      $(d, v) \leftarrow extractMin(pq)$ 
6:     если  $d > dist[v]$  то
7:       continue
8:     конец если ▷ устаревшая запись
9:     для each  $(v, u, w) \in E$  выполнять
10:      если  $dist[v] + w < dist[u]$  то ▷ релаксация
11:         $dist[u] \leftarrow dist[v] + w$ 
12:         $pq.push((dist[u], u))$ 
13:      конец если
14:    конец для
15:  конец пока
16: конец процедура
```

Релаксация сохраняет инвариант: $dist[v]$ — длина некоторого пути к v . Жадный выбор минимального $dist[v]$ гарантирует оптимальность при неотрицательных весах.

Алгоритм Беллмана-Форда (любые веса)

```
1: процедура BELLMANFORD( $G, s$ )
2:   для each  $v \in V$  выполнять
3:      $dist[v] \leftarrow \infty$ 
4:   конец для
5:    $dist[s] \leftarrow 0$ 
6:   для  $i \leftarrow 1$  to  $|V| - 1$  выполнять
7:     для each  $(u, v, w) \in E$  выполнять
8:       если  $dist[u] + w < dist[v]$  то
9:          $dist[v] \leftarrow dist[u] + w$ 
10:      конец если
11:    конец для
12:  конец для
13:  для each  $(u, v, w) \in E$  выполнять
14:    если  $dist[u] + w < dist[v]$  то
15:      вернуть "Отрицательный цикл найден"
16:    конец если
17:  конец для
18:  вернуть  $dist$ 
19: конец процедура
```

► Проверка на отрицательный цикл

Сложность: $O(V \cdot E)$. Работает с отрицательными весами (без отрицательных циклов).

Алгоритм Косарайю (компоненты сильной связности)

```
1: процедура KOSARAJU( $G$ )
2:
3:    $stack \leftarrow \emptyset$ 
4:   для each  $v \in V$  выполнять
5:     если not  $visited[v]$  то DFS1( $v, stack$ )
6:     конец если
7:   конец для
8:
9:
10:   $scc\_count \leftarrow 0$ 
11:  пока  $stack$  не пуст выполнять
12:     $v \leftarrow pop(stack)$ 
13:    если not  $assigned[v]$  то
14:      DFS2( $v, scc\_count, G^T$ )
15:       $scc\_count \leftarrow scc\_count + 1$ 
16:    конец если
17:  конец пока
18: конец процедуры
```

▷ Шаг 1: порядок завершения DFS

▷ Шаг 2: построить обратный граф G^T

▷ Шаг 3: DFS на G^T в порядке $stack$

Сложность: $O(V + E)$. Требуется два прохода и обратный граф.

Алгоритм Тарьяна — компоненты сильной связности (SCC)

Идея: один DFS, нумеруем вершины в порядке обхода (*index*), вычисляем $low[v]$ — наименьший индекс вершины, достижимой из поддерева v (включая обратные и перекрёстные рёбра).

Definition

$$low[v] = \min \begin{cases} index[v] \\ index[u] \text{ для любого обратного/перекрёстного ребра } v \rightarrow u \text{ с } u \text{ в стеке} \\ low[u] \text{ для любого ребра дерева } v \rightarrow u \end{cases}$$

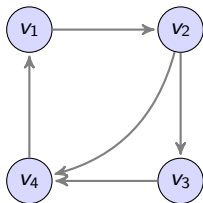
Вершина v является корнем SCC (началом компоненты) тогда и только тогда, когда:

$$low[v] = index[v]$$

Алгоритм Тарьяна — псевдокод и пояснения

```
1: процедура TARJAN( $v$ )
2:    $index[v] \leftarrow low[v] \leftarrow currIndex$ ;  $currIndex \leftarrow currIndex + 1$ 
3:    $stack.push(v)$ ;  $onStack[v] \leftarrow true$ 
4:   для each  $u \in neighbours(v)$  выполнять
5:     если  $index[u]$  не определён то                                     ▷ ребро дерева
6:       TARJAN( $u$ )
7:        $low[v] \leftarrow \min(low[v], low[u])$ 
8:     иначе если  $onStack[u]$  то                                         ▷ обратное или перекрёстное ребро в текущую SCC
9:        $low[v] \leftarrow \min(low[v], index[u])$ 
10:    конец если
11:  конец для
12:  если  $low[v] = index[v]$  то                                           ▷  $v$  — корень SCC
13:    ▷ Извлекаем все вершины до  $v$  включительно
14:    повторять
15:       $u \leftarrow stack.pop()$ ;  $onStack[u] \leftarrow false$ 
16:       $scc[u] \leftarrow sccCount$ 
17:    пока не  $u = v$ 
18:     $sccCount \leftarrow sccCount + 1$ 
19:  конец если
20: конец процедура
```

Сложность: $O(V + E)$ — каждый узел и ребро просматриваются один раз.



Пример:

- Порядок DFS: $v_1(1)$, $v_2(2)$, $v_3(3)$, $v_4(4)$.
- Обратное ребро $v_4 \rightarrow v_1$: $low[4] = \min(4, 1) = 1$, передаётся вверх.
- $low[3] = \min(3, low[4] = 1) = 1$, $low[2] = 1$, $low[1] = 1$.
- Все вершины в одной SCC, так как $low[1] = index[1] = 1$.

Примеры NP-трудных задач:

- Задача коммивояжёра (TSP)
- Максимальная клика
- Минимальное вершинное покрытие
- Раскраска графа

Целочисленное линейное программирование (ILP) для вершинного покрытия:

$$\begin{aligned} \min \quad & \sum_{v \in V} x_v \\ \text{при} \quad & x_u + x_v \geq 1 \quad \forall (u, v) \in E \\ & x_v \in \{0, 1\} \end{aligned}$$

Релаксация $0 \leq x_v \leq 1$ даёт нижнюю границу.

Метод ветвей и границ (Branch and Bound)

Общая схема для минимизации:

- ❶ **Ветвление (Branching)**: выбираем переменную x_i , создаём две подзадачи: $x_i = 0$ и $x_i = 1$.
- ❷ **Оценка (Bounding)**: решаем релаксацию (LP) $\rightarrow$ нижняя граница LB .
- ❸ **Отсечение (Pruning)**: если $LB \geq$ лучшее найденное решение (верхняя граница UB), отбрасываем.

```
1: процедура ВнВ(problem, UB)
2:   если problem тривиален то
3:     обновить UB, вернуть
4:   конец если
5:    $LB \leftarrow$  релаксация(problem)
6:   если  $LB \geq UB$  то
7:     вернуть
8:   конец если
9:   выбрать переменную  $x$  для ветвления
10:  ВнВ(problem  $\cup$   $\{x = 0\}$ , UB)
11:  ВнВ(problem  $\cup$   $\{x = 1\}$ , UB)
12: конец процедура
```

Зависимость эффективности от порядка обхода дерева

DFS (глубина): - Малая память - Быстро находит решение - Риск долгой неперспективной ветви

BFS (ширина): - Гарантия оптимальности - Экспоненциальная память

Best-first (лучшая граница): - Выбираем узел с мин. LB - Обычно быстрее всего - Требуется приоритетная очередь

На практике часто используют комбинации: сначала DFS для получения хорошей UB , затем Best-first.